

ResQ-Chain

Hyper-Local Food Waste Rescue Platform
AI-Driven Dynamic Markdown & Real-Time Redistribution Engine

Hackathon Strategy & Technical Playbook
Prepared as a judge-and-developer-style deep dive covering problem framing, market positioning, architecture, ML design, data strategy, notification logic, implementation roadmap, and live-demo plan.

Domain	Hyper-local food waste / retail-tech / social impact
Core Stack	Python (FastAPI), Scikit-Learn, PostgreSQL + PostGIS, Redis, Celery, React, Twilio
Document Scope	17-point technical & strategic breakdown, chapter-wise
Companion File	RULE.md — agent/developer working rules for phase-based build

Table of Contents

01	The Problem — Who Suffers, and What They Expect
02	Our Solution & Market Significance
03	Competitive Landscape — What Others Are Doing
04	Technical Overview of the Entire Project
05	Architecture — Client Side & Server Side
06	Technology Stack
07	Workflows — Customer, Store Staff & Developer
08	Implementation Roadmap (Phase-by-Phase, Time-Boxed)
09	Innovative Angles & Helpful Third-Party APIs
10	Personalized Notification Channel Strategy
11	The Machine Learning Model(s)
12	Data Requirements — In Detail
13	Parameters, Training/Testing Strategy & Algorithms
14	The Ecosystem — Integration & Pitch Framing
15	Demo Plan — Live Run-of-Show
16	Companion Deliverable — RULE.md (Agent Working Rules)

CHAPTER 1

The Problem — Who Suffers, and What They Expect

1.1 The scale of the problem

Roughly a third of all food produced for human consumption globally is lost or wasted every year — on the order of 1.3 billion tonnes — and the figure has stayed stubbornly consistent even as awareness has grown. In the United States alone, the retail sector discards an estimated 43 billion pounds (about 19.5 million metric tons) of food annually, and only a sliver of that — roughly 1.2 billion pounds — is donated. In the United Kingdom, supermarkets alone generate around 200,000 tons of food waste a year; in the European Union, retail and wholesale account for roughly 40% of all food waste; in India, retail-level waste is estimated at 5–16 million tons annually. Wasted retail food is not a rounding error either — its value is estimated at roughly twice the profit retailers make from food sales in the first place, because the sunk cost of procurement, labor, refrigeration, and shelf space is lost the moment an item is thrown out.

1.2 Why it keeps happening — the root causes

- **Static, manually-triggered discounts.** Most stores still follow a fixed playbook — e.g. "mark down 20% four days out, 40% two days out" — regardless of how fast an item is actually selling, local weather, or foot traffic that day. This is a blunt instrument dating back decades.
- **Blind to demand signals.** Discounting decisions rarely account for real-time variables like a sudden heatwave killing dairy foot-traffic, a rainy evening cutting store visits, or a weekend spike in demand that means a markdown wasn't even necessary.
- **Labor-intensive, department-by-department decisions.** Markdown calls are typically made manually by department managers, which doesn't scale, is inconsistent across shifts/stores, and competes with already-stretched staff time.
- **Disconnected redistribution.** Even when stores want to donate surplus, there's often no real-time channel to nearby shelters — donation coordination happens through phone calls, inconsistent pickup windows, and liability concerns around "un-donatable" surplus, so recoverable food still gets bagged for the compactor.
- **No proactive channel to nearby buyers.** Even highly motivated bargain shoppers and community organizations have no way of knowing, at the moment surplus appears, that it exists a few minutes away.

1.3 How stores suffer

- Shrinkage eats directly into thin grocery margins (perishables already run some of the lowest margins in retail).
- Manual markdown workflows consume staff hours that could go toward customer-facing work.
- Poor visibility into *why* an item isn't moving (price vs. weather vs. just low footfall) means the same mistakes repeat weekly.
- Missed ESG/CSR reporting opportunities — waste reduction is increasingly a boardroom and investor metric, but most stores have no structured data trail to report on it.

1.4 How customers and communities suffer

- Price-sensitive shoppers have no visibility into real, deep, time-boxed discounts happening near them right now — clearance sections are hit-or-miss and location-dependent.
- Shelters and NGOs operate on unpredictable, manually-coordinated supply, making meal planning difficult and often missing the pickup window entirely.

- Underserved users without smartphones, data plans, or app literacy are structurally excluded from every existing app-first solution.

1.5 What each side actually expects from a solution

Stakeholder	What they expect
Grocery / retail stores	Automated, low-effort markdown decisions that protect margin, reduce manual labor, and produce reportable impact metrics — without a costly POS overhaul.
Price-sensitive consumers	Real-time, trustworthy alerts about steep, genuine discounts nearby, with a frictionless way to claim (ideally without installing yet another app).
Shelters / NGOs	Predictable, prioritized, and free/subsidized access to recoverable surplus, with advance notice instead of one-off scrambles.
Municipalities / ESG stakeholders	Verifiable data on diverted food, emissions avoided, and community impact that can back up sustainability reporting and policy targets (e.g. SDG 12.3).

Assumption flagged: Where the source material didn't specify a number (e.g. exact hackathon duration, target city/region), this document states the assumption explicitly rather than guessing silently — see Chapter 8.

CHAPTER 2

Our Solution & Market Significance

2.1 What ResQ-Chain actually is

ResQ-Chain is a closed-loop, hyper-local liquidation-and-redistribution engine that sits directly on top of a store's inventory data. It continuously scores every perishable batch against its expiry timeline, then layers in external signals — weather, historical demand, foot traffic — through a machine learning model to decide two things a static system can't: *how much* to discount and *who* to target first (general consumers vs. B2B/NGO bulk buyers). When a batch crosses a critical threshold, the system automatically fires a targeted SMS/WhatsApp flash-sale broadcast within a geofenced radius, takes payment, and routes fulfillment as pickup or delivery — while shelters on standing orders get subsidized, no-payment priority access.

2.2 What makes it structurally different

- **Predictive, not reactive, pricing.** Discounts are computed from a live model, not a fixed markdown calendar — the same yogurt might get a smaller markdown on a busy Saturday and a steeper one during a rainy Tuesday lull.
- **No-app-required distribution.** SMS/WhatsApp-first claiming (including reply-"YES" claiming) removes the single biggest adoption barrier every competing app faces: getting a stranger to install something for a one-off discount.
- **Dual-sided targeting in one engine.** The same ML layer that reprices for consumers also decides when a batch is better routed to a B2B/NGO buyer — most competitors pick one lane (consumer marketplace or pricing infrastructure) and stay there.
- **Closes the loop end-to-end.** Detection → pricing → alert → reservation → payment → fulfillment → impact reporting all live in one pipeline, rather than requiring a store to bolt together a pricing tool, a donation hotline, and a separate marketplace app.

2.3 Market significance

The category ResQ-Chain sits inside is large and compounding. Independent market analyses put the global food-waste-management market at roughly USD 82–87 billion in 2026, on a path toward the USD 130 billion range by the early-to-mid 2030s across most estimates (growth rates cited between ~5% and ~11% CAGR depending on methodology and scope). Adjacent to that, the broader zero-waste / sustainable-grocery market is estimated at roughly USD 313 billion in 2026, growing at a ~7.75% CAGR. On the demand side, retailers using AI-driven forecasting and markdown tools have reported average food-waste reductions in the range of 15–40% per store in published case studies, alongside single-digit-to-double-digit revenue recovery gains — proof that this is not a purely philanthropic pitch, it is also a margin-recovery story, which is what gets a retail buyer to actually sign a contract.

Why this matters for the pitch: Judges and retail buyers respond to a two-sided ROI story: "we cut your shrinkage" (a cost problem) sits next to "we generate goodwill and ESG data" (a reputation problem) — ResQ-Chain's dynamic pricing plus impact dashboard hits both in one product, which is the strongest structural argument for market fit.

CHAPTER 3

Competitive Landscape — What Others Are Doing

3.1 The existing players, mapped

Food-waste tech roughly splits into three lanes today, and almost nobody occupies all three at once.

Player	Lane	Model	Notable data point
Too Good To Go	Consumer marketplace	Paid "surprise bags" of unsold food, app-only, restaurant + retail supply, 19+ countries	350M+ meals rescued globally
Flashfood	Consumer marketplace	Itemized (not surprise) discounted groceries via app, partnered grocery banners (Loblaws, Meijer, Stop & Shop, etc.)	100M+ lbs diverted, \$247M+ saved by shoppers
Olio	Community sharing	Free neighbor-to-neighbor food sharing, no payment layer	Free / peer-to-peer model, different demand entirely
Wasteless	Pricing infrastructure (B2B)	True AI dynamic pricing tied to electronic shelf labels / POS, store-facing only, no consumer alert layer	Reported ~33–40% waste reduction in pilot stores
Afresh / Shelf Engine	Forecasting & ordering (B2B)	AI demand forecasting to prevent over-ordering upstream; doesn't touch markdown liquidation	~14.8% average waste reduction per store in published case studies
ResQ-Chain	All three, connected	Predictive dynamic pricing + hyper-local SMS/WhatsApp broadcast + reservation/payment + NGO standing orders in one pipeline	—

3.2 Where ResQ-Chain has real whitespace

- **vs. Too Good To Go / Flashfood:** both are fundamentally app-first marketplaces with human- or rule-set discounts on the store side — neither runs a live, weather-and-demand-aware ML repricing engine underneath. ResQ-Chain's markdown is computed, not chosen.
- **vs. Wasteless:** genuinely the closest technical cousin on the pricing side, and honestly the strongest existing proof-point that AI dynamic markdown works (pilot data shows real waste and revenue gains). But Wasteless is infrastructure sold into a store's POS/ESL stack — it has no consumer-facing broadcast, no SMS/no-app claim flow, and no NGO/shelter standing-order layer. ResQ-Chain could reasonably be pitched as "what Wasteless would look like if it also owned the last mile."

- **vs. Afresh / Shelf Engine:** these solve the upstream over-ordering problem, not the downstream liquidation-and-redistribution problem. Different stage of the supply chain entirely — genuinely complementary rather than competitive.
- **vs. Olio:** free peer-to-peer sharing has no monetization and no store-integration depth; it solves a different (household-level) waste problem.

3.3 Honest caveats to be ready for

- Distribution and retailer relationships are TGTG's and Flashfood's real moat, built over roughly a decade — a hackathon build cannot claim to out-distribute them. The honest pitch angle is technical differentiation and an underserved-market wedge (accessibility, SMS-first, emerging markets), not scale.
- Dynamic pricing at the shelf-edge (Wasteless's approach) faces a real integration barrier: most retailers still lack high-quality real-time inventory systems or electronic shelf labels, which is exactly why ResQ-Chain's MVP should target a lightweight ingestion path (CSV/photo/manual entry) rather than assuming deep POS integration on day one — see Chapter 8.
- "87% of retail food waste comes from expiry-driven disposal" (a figure cited by dynamic-pricing vendors) is a useful stat for the pitch, but it should be framed as an industry estimate, not an audited figure.

CHAPTER 4

Technical Overview of the Entire Project

At the system level, ResQ-Chain is best understood as three cooperating planes rather than one monolith. Keeping this separation explicit — even inside a single hackathon repo — is what lets each phase in Chapter 8 be built and demoed independently.

Plane	Responsibility	Core components
1. Data & Prediction plane	Ingests inventory/batch data, enriches it with weather & demand history, runs the ML markdown engine, decides price + target segment.	PostgreSQL + PostGIS, Scikit-Learn model (joblib), Celery beat scheduled scans
2. Orchestration plane	Owns business logic: reservations, locking, payment state machine, standing-order routing, reliability scoring.	FastAPI, Redis (cache + broker + TTL locks), Celery workers
3. Delivery plane	Everything the human on the other end sees or receives.	Twilio (SMS + WhatsApp), Stripe/Razorpay checkout, React claim page & dashboards, WebSocket live-price feed

4.1 The event loop, in one sentence

A scheduled scan finds at-risk batches → the model prices and segments them → if a threshold is breached, an alert fires to a geofenced audience → a claim reserves stock with a TTL lock → payment confirms the reservation → fulfillment (pickup/delivery) executes → the outcome (sold, expired, donated) feeds back into the training data for the next model refresh.

CHAPTER 5

Architecture — Client Side & Server Side

5.1 Client side

- **Public Claim Page** (React + Tailwind, no login) — opened from an SMS/WhatsApp link or a scanned QR code; shows item, discounted price, countdown timer on the reservation, pickup-vs-delivery toggle, and (for auction batches) a live descending price fed over WebSocket.
- **Store / Admin Dashboard** (React + Tailwind, authenticated) — inventory view, batch status, manual override controls, and the Eco-Impact Analytics view (dollars saved, lbs rescued, CO2e avoided) built with a charting library on top of the same PostGIS/Postgres data.
- **NGO / Shelter Portal** — a lightweight authenticated view for managing Standing Order rules ("always prioritize dairy & bread over 5kg, notify first for anything expiring within 6 hours").
- **Map layer** — Leaflet.js (per the latest stack) with OpenStreetMap tiles for the geofenced radius visualization; Mapbox GL JS remains a drop-in upgrade path if richer styling is needed later.

5.2 Server side

- **FastAPI** — REST endpoints for inventory ingestion, claim creation, payment webhooks, standing-order CRUD, and a WebSocket endpoint for the live auction price feed.
- **Celery + Redis** — Celery beat runs the periodic expiry-risk scan and nightly model retraining; Celery workers handle SMS/WhatsApp dispatch, OCR ingestion jobs, and delivery-fee calculation so none of that blocks the request/response cycle. Redis doubles as the short-lived *reservation lock* store (a `reserved\_until` TTL key per batch) and as the Celery broker.
- **PostgreSQL + PostGIS** — system of record for stores, batches, users, claims, and standing orders; PostGIS handles the radius/distance queries that power geofenced targeting and delivery-fee logic.
- **ML Inference** — the Scikit-Learn model is trained offline/nightly and loaded via Joblib inside a Celery task (or a small internal FastAPI route) so scoring a batch is a fast, synchronous lookup at scan time, not a live training run.
- **Twilio** — outbound SMS/WhatsApp dispatch plus an inbound webhook receiver for "reply YES" claiming.
- **Stripe / Razorpay** — hosted checkout link generation plus a webhook receiver that transitions a reservation into a confirmed, paid claim.

5.3 Request flow (text diagram)

```

STORE INVENTORY (CSV / photo-OCR / manual entry)
|
v
[FastAPI ingestion endpoint] --> PostgreSQL (batches table)
|
v
[Celery beat: expiry-risk scan, every N minutes]
|
v
[ML scorer] <-- weather API, demand history, foot-traffic index
| (price, segment: B2C / B2B-NGO, urgency tier)
v
threshold breached? ----no----> re-check next scan
| yes
v
[Celery worker: build geofenced audience via PostGIS radius query]
|
v
[Twilio dispatch: SMS / WhatsApp] -----> end user's phone
|                                     |
|                                     taps link OR replies "YES"
v                                     v
[Redis: reserved_until TTL lock set] <---- [FastAPI claim endpoint]
|
v
[Stripe/Razorpay checkout link] --> user pays --> [webhook]
|
v
[Claim CONFIRMED] --> pickup (QR code) or delivery (courier/volunteer queue)
|
v
[Outcome logged: sold / expired / donated] --> feeds nightly retrain + impact dashboard

```

CHAPTER 6

Technology Stack

6.1 Confirmed stack

Layer	Technology	Why
API & backend	Python — FastAPI	Async-friendly, fast to scaffold, auto-generated OpenAPI docs are useful for a judge Q&A walkthrough.
ML	Scikit-Learn + Joblib	Fast to train/iterate within a hackathon window; Joblib gives cheap model persistence/reload for inference.
Database	PostgreSQL (+ PostGIS)	Relational integrity for transactional claims + native geospatial radius queries for geofencing, in one engine.
Async/jobs	Redis + Celery	One tool doing double duty: message broker for background jobs and TTL-based reservation locking to prevent double-claims.
Comms	Twilio API (SMS + WhatsApp Business API)	Single vendor for both channels, inbound webhook support enables "reply YES" claiming.
Frontend	React.js + Tailwind CSS	Fast component iteration, easy to make the claim page genuinely lightweight/mobile-first.
Mapping	Leaflet.js (OpenStreetMap)	Free tile usage with no API key friction for a hackathon demo; Mapbox GL JS is a documented upgrade path if styling budget allows.

6.2 Additions worth scoping in (see Chapter 9 for the full rationale)

- **Payments:** Stripe or Razorpay test mode (Razorpay is the natural default for an India-facing demo, pairing with WhatsApp Business API which has much higher engagement in that market).
- **Weather:** OpenWeatherMap or Tomorrow.io free tier for the real-time/forecast variable feeding the ML model.
- **OCR / barcode:** Tesseract (open-source) or a barcode-scanning JS library (e.g. a ZXing-based package) for "zero-effort" store onboarding.
- **Geocoding:** OpenStreetMap Nominatim (free) if store/user addresses need to be geocoded rather than captured as raw coordinates.

CHAPTER 7

Workflows — Customer, Store Staff & Developer

7.1 Customer workflow

- Receives SMS/WhatsApp alert for a nearby discounted item (or a predictive "heads-up" pre-alert).
- Taps the tokenized claim link (or replies "YES" directly to the text) — no account, no app install.
- Item is reserved with a short countdown timer visible on the claim page.
- Completes secure checkout (Stripe/Razorpay), then chooses self-pickup or delivery.
- Pickup: receives a QR code for in-store collection. Delivery: sees tracked status (pending → out for delivery → delivered).
- After fulfillment, receives a shareable "impact receipt" and their reliability score updates.

7.2 Store staff workflow

- Onboards a batch by CSV upload, barcode scan, or a shelf-label photo (OCR pre-fills expiry/product details) — deliberately zero-friction to remove the #1 real-world adoption barrier.
- Sees at-risk batches surfaced automatically on the dashboard with the model's suggested price and target segment; can accept or manually override.
- Gets visibility into which alerts converted, which expired unclaimed, and rolling waste/revenue-recovery metrics.

7.3 Developer workflow (ties directly into RULE.md, Chapter 16)

- Work happens strictly phase by phase, each on its own branch, with frequent small commits.
- Before pushing code or starting a new phase, the developer/agent explicitly requests permission.
- STATUS.md and DECISIONS.md are updated at every commit/phase boundary so a new contributor (human or agent) can resume with minimal ramp-up context.
- Local environment: docker-compose spinning up Postgres+PostGIS, Redis, and the FastAPI app; ``.env.example`` checked in (never real secrets); seed script populates demo stores/batches so the app is demo-ready from a clean clone.

CHAPTER 8

Implementation Roadmap (Phase-by-Phase, Time-Boxed)

Assumption stated up front: The source material says "time is short" without naming an exact window. This roadmap assumes a typical **36-hour hackathon** (build time, excluding sleep/breaks) as a reasonable default — every phase below has a duration you can compress or stretch proportionally if your actual window is 24h, 48h, or longer.

8.1 Phase table

Phase	Duration	Deliverable	Cut-scope fallback if behind schedule
0 — Setup	2h	Repo, docker-compose, DB schema, branch/CI skeleton, seed data script	Skip CI; seed data by hand
1 — Inventory Core	4h	FastAPI CRUD for stores/batches, Postgres schema live, CSV/manual ingestion endpoint	Manual entry only, skip OCR for now
2 — Markdown Engine v1	4h	Rule-based pricing first, then swap in the trained Scikit-Learn model behind the same interface	Ship the rule-based version if the ML model isn't converging in time — still demoable
3 — Geofencing & Claims	5h	PostGIS radius queries, tokenized claim link, Redis reservation TTL lock	Fixed radius per store if dynamic radius tuning is too slow to build
4 — Comms	4h	Twilio SMS dispatch + inbound webhook for "reply YES"; WhatsApp as stretch	SMS-only if WhatsApp Business API sandbox approval is delayed
5 — Payments & Fulfillment	4h	Stripe/Razorpay test-mode checkout + webhook, pickup vs. delivery toggle, QR code generation	Mock/skip real courier routing — a static "volunteer queue" list is enough for a demo
6 — Dashboard & Polish	5h	Eco-Impact dashboard, shareable impact receipt, standing-order UI for NGOs, descending-price auction view	Auction view is the first thing to cut if time runs out — it's a nice-to-have, not core
7 — Demo Rehearsal	4h	Seeded demo dataset, run-through, fallback recording, Q&A prep	—

Total: ~32 hours of the 36-hour budget, leaving a buffer for the inevitable integration friction (Twilio sandbox approval, PostGIS setup, payment webhook debugging tend to eat the most unplanned time).

8.2 Priority order if the timeline compresses further

If time is cut to, say, 18–24 hours: build Phases 0–3 fully (this alone demonstrates the core differentiator — predictive pricing + geofenced claim), do a minimal version of Phase 4 (SMS only, outbound only, skip inbound reply-claim), skip real payment integration in favor of a mocked "Pay (Demo Mode)" button, and spend remaining time on Phase 7 rehearsal. A working narrow slice beats a broken wide one in front of judges.

CHAPTER 9

Innovative Angles & Helpful Third-Party APIs

9.1 APIs that remove real build time

Need	Suggested API/tool	Why it helps a short build
Weather signal	OpenWeatherMap or Tomorrow.io (free tier)	Drop-in current + forecast data keyed by lat/long — directly feeds the ML model's weather features.
Geocoding	OpenStreetMap Nominatim	Free, no key friction, enough accuracy for store/user coordinates in a demo.
OCR for shelf labels	Tesseract (open-source, local)	No API key or rate limit to worry about mid-demo; good enough accuracy for a controlled hackathon environment.
Messaging	Twilio Programmable Messaging + WhatsApp Business API sandbox	Already in the stack; sandbox mode is enough for a live demo without a full business verification.
Payments	Stripe test mode / Razorpay test mode	Full checkout + webhook flow with fake cards — looks completely real to a judge without touching live money.
Carbon/impact conversion	Public USDA/EPA-style food-waste-to-CO2e conversion factors	Lets the impact dashboard show a credible "kg CO2e avoided" number without inventing methodology.
Copy generation	A small LLM call (e.g. the Claude API) to auto-draft the SMS/WhatsApp alert text and the shareable impact receipt caption	Turns a static template into personalized, on-brand copy per alert — and doubles as a nice "AI-native" demo beat for judges.

9.2 Other innovative angles worth considering

- **Reliability-weighted alert routing** (already in the spec) — worth demoing explicitly, since "claimed but never picked up" is the most common complaint about existing food-rescue apps.
- **Local events/holiday calendar awareness** — a public holiday calendar API can flag demand spikes (e.g. a festival weekend) so the model doesn't over-discount when foot traffic is about to surge anyway.
- **Neighborhood leaderboard / gamification** — a lightweight "top rescuers this week" leaderboard per geofenced zone, built entirely from data already being collected, is a cheap way to add a viral/retention hook without new infrastructure.
- **"Claudeception" copy + insight layer** — beyond SMS copy, an LLM call can summarize a store's weekly waste patterns into a plain-English recommendation ("Wednesdays consistently over-stock dairy — consider reducing standing orders by ~8%"), turning the analytics dashboard from a chart into an advisor.

Personalized Notification Channel Strategy

10.1 Signals used to identify interested, reachable users

- **Category affinity** — opt-in preferences plus what a user has actually claimed before (dairy, produce, bakery, meat, etc.), so alerts aren't broadcast to everyone within radius regardless of relevance.
- **Reliability score** — claim-to-pickup ratio; higher-reliability users get priority routing on short-fuse, high-urgency batches where a no-show is costliest.
- **Response latency history** — how quickly a user typically acts on an alert, used to decide who gets first crack at very short claim windows.
- **Channel engagement history** — whether a user has historically tapped WhatsApp rich-card buttons vs. plain SMS links vs. replying "YES" by text; this is the strongest signal for which channel to use next time.
- **Price sensitivity** — the minimum discount depth that has triggered a claim historically, useful for deciding who to notify first when a batch is only lightly discounted.
- **Geo-distance / travel time** — tighter radius bands (e.g. 5-minute vs. 10-minute) get priority ordering since they're most likely to actually complete the pickup in time.
- **Standing-order status** — NGOs/shelters are a separate, always-priority queue independent of the general scoring above.

10.2 Segment → channel decision logic

Segment	Routing behavior
Standing-order NGO/shelter	Bypasses general broadcast entirely; notified first, subsidized/free, on their own priority channel regardless of reliability score.
High-reliability, fast responder, within tight radius	Priority WhatsApp rich card (image + one-tap claim button) if they've engaged with WhatsApp before; otherwise SMS link.
Mid-tier / general consumer	Standard SMS with claim link; WhatsApp if available and previously used.
Historically low link-click, high reply-rate users	SMS reply-"YES" claiming prioritized over a tappable link — an accessibility-driven default, not a downgrade.
Low-engagement / cold users	Only included on high-volume or near-total-loss batches, to avoid alert fatigue and opt-outs.

10.3 Guardrails

- Per-user daily notification cap, independent of how many nearby batches trigger alerts.
- Category opt-in is a hard filter, applied before any scoring — never override a user's stated preferences even for a high-urgency batch.
- Every alert includes a one-tap/one-reply opt-out to keep the channel trustworthy and compliant.

The Machine Learning Model(s)

11.1 Two coupled models, not one

Model A — Markdown / Price Optimization

A regression model (start with Scikit-Learn's `GradientBoostingRegressor` or `HistGradientBoostingRegressor` for speed) predicts expected sell-through probability for a given discount depth, category, and context (weather, time-to-expiry, velocity). A small discrete search then picks the discount level (e.g., stepping through 0%, 10%, ... 70%) that maximizes *expected recovered revenue* (probability of sale × discounted price), subject to a hard constraint that forces maximum discount once hours-to-expiry drops below a critical threshold.

Model B — Segment / Channel Classifier

A lighter classifier (logistic regression or a small decision tree) decides whether a batch is better routed to B2C consumers or B2B/NGO bulk buyers, using batch size, category, and historical B2B demand for that category — bulk perishables with low retail sell-through but strong shelter demand should route B2B-first rather than fragmenting into many small consumer claims.

Model C (stretch) — Predictive Pre-Alert

A simple classifier estimating the probability that a discount will trigger within the next N hours, using current velocity and the weather trend, powers the "heads-up" pre-alert feature (e.g., "milk likely to be discounted around 6 PM tonight"). This does not need a true time-series model like Prophet for a hackathon build — a well-featured gradient-boosted classifier is enough.

11.2 Cold start & fallback

New stores/SKUs with no sales history won't have enough data for the model to be confident. In that case the system should fall back to a category-level average shelf-life/elasticity curve (a simple lookup table), with the model taking over once a few weeks of real data accumulate. This fallback should be explicit and visible on the dashboard, not silent — both for trust and for the demo narrative ("here's how the system behaves on day one vs. after learning").

CHAPTER 12

Data Requirements — In Detail

12.1 Store inventory / batch data

- SKU, product name, category, batch ID
- Quantity on hand
- Unit cost and current retail price
- Expiry / best-by date and received date
- Shelf/section location within store

12.2 Historical sales / demand data

- Daily units sold per SKU per store
- Price at time of each sale (to reconstruct elasticity)
- Day-of-week / hour-of-day of sale
- Any promotion flag active at time of sale

12.3 Weather data (external)

- Current temperature and precipitation at store location
- 24–48h forecast (temperature, precipitation probability)
- Historical weather for the training window, matched to store geo-coordinates

12.4 Foot-traffic / demand-context proxy

- Hourly POS transaction counts per store (if available) as a foot-traffic proxy
- Local events/holiday calendar flags

12.5 Customer / user data (privacy-conscious by design)

- Coarse geolocation (opt-in) for geofencing — not continuous tracking
- Claim history: what was claimed, when, and whether pickup/delivery completed
- Reliability score inputs (claimed vs. actually picked up)
- Channel engagement history (SMS vs. WhatsApp vs. reply-claim)
- Category opt-ins and communication preferences
- Payment method — tokenized only via Stripe/Razorpay; raw card data should never touch this system's own database (PCI scope stays with the payment processor).

12.6 NGO / shelter data

- Standing-order rules (category, minimum quantity, priority time window)
- Verified organization status
- Delivery vs. pickup capability

12.7 Reference / derived data

- Category-level average shelf-life curves, for cold-start pricing before enough item-level history exists
- Public waste-to-CO2e conversion factors (e.g. USDA/EPA-style constants), for the impact dashboard's emissions-avoided metric

CHAPTER 13

Parameters, Training/Testing Strategy & Algorithms

13.1 Feature list (Model A — pricing)

Feature	Type	Notes
time_to_expiry_hours	Numeric	Core urgency driver
category	Categorical	One-hot or target-encoded
current_price / cost_price	Numeric	Establishes margin floor
qty_on_hand	Numeric	Volume to liquidate
avg_daily_velocity_7d / 28d	Numeric	Recent vs. baseline demand
temperature_today, precip_probability	Numeric	From weather API
is_weekend, is_holiday, hour_of_day	Boolean/Numeric	Temporal demand context
store_foot_traffic_index	Numeric	POS-transaction-based proxy
past_discount_depth, past_sellthrough_rate	Numeric	Item/category historical response
local_demand_score	Numeric	Distance-weighted nearby claim activity
b2b_flag	Boolean	Whether batch qualifies for bulk/B2B routing

13.2 Target variable

Probability of sale (or units sold) within the remaining time-to-expiry window, given a candidate discount depth — this is what Model A is trained to predict, and the price-search step then optimizes over it.

13.3 Train/test split methodology

Use a **time-based split**, not a random split — train on earlier weeks and validate on the most recent week(s). Retail demand data is time-series-like and has drift (seasonality, promotions), so a random split would leak future information into training and overstate accuracy.

13.4 Algorithms, by stage

Stage	Algorithm	Why
Baseline	Linear / Logistic Regression	Fast, interpretable, a sanity-check floor before investing in anything heavier.
Production candidate	Gradient Boosted Trees (Scikit-Learn GradientBoostingRegressor / HistGradientBoostingRegressor, or LightGBM/XGBoost if time allows)	Handles non-linear interactions (e.g. weather × category) well, trains fast enough for a hackathon iteration loop.

Stage	Algorithm	Why
Segment classifier (Model B)	Logistic Regression or a small Decision Tree	Small feature set, interpretability matters more than raw accuracy here for the pitch narrative.
Evaluation — regression	MAE / RMSE on predicted sell-through / expected revenue	Standard, explainable to non-ML judges.
Evaluation — classifier	AUC / F1	Handles the class imbalance typical of "did this batch get discounted" style labels.

13.5 Handling the lack of real historical data

For a hackathon build, generate a **synthetic dataset** with realistic per-category price-elasticity curves plus noise, seeded for reproducibility. This should be stated openly to judges as synthetic — transparency about the data source is more credible than implying real production data that doesn't exist yet. Document the generation logic in the repo so it can later be swapped for a real retailer feed without changing the model interface.

13.6 Retraining cadence

Nightly batch retrain via Celery beat once enough real/simulated data accumulates, with the rule-based fallback (Chapter 11.2) automatically active whenever model confidence or available data volume for a given SKU/category is too low.

The Ecosystem — Integration & Pitch Framing

14.1 The full stakeholder ecosystem

- **Grocery chains & independent stores** — the supply side; ingestion starts lightweight (CSV/photo/manual) and can grow toward direct POS integration once trust is established.
- **Consumers** — the primary B2C demand side, reached without requiring an app install.
- **NGOs / shelters** — the subsidized B2B demand side, served through standing orders.
- **Delivery / volunteer network** — fulfillment for the delivery option; starts as a simple queue, can later integrate a courier API.
- **Payment processors** — Stripe/Razorpay, handling PCI scope and the revenue-split logic (store majority share, small platform fee, delivery fee routed to couriers).
- **Municipalities / ESG stakeholders** — a long-term monetizable layer: aggregated, anonymized impact data exported for sustainability reporting or policy tracking (e.g., against SDG 12.3 targets).

14.2 Integration path over time

- **MVP:** manual/CSV/photo inventory ingestion at a small number of pilot stores — proves the pricing and broadcast loop without requiring any retailer IT investment.
- **Growth:** direct POS/inventory-system plugin integrations with grocery banners, mirroring how Flashfood scaled through named retail partnerships rather than a fully self-serve model.
- **Maturity:** municipal/NGO-network partnerships for verified shelter onboarding at scale, plus an ESG/CSR reporting export product for retailers — a second, data-driven revenue line beyond transaction fees.

14.3 Pitch structure

Problem (Chapter 1) → Solution (Chapter 2) → Live demo (Chapter 15) → Differentiation vs. named competitors (Chapter 3) → Business model (revenue split + subsidized NGO access, Chapter 2 & the original write-up's payment layer) → Impact metrics (the Eco-Impact dashboard, live on screen) → The ask (pilot store partnership / seed investment / hackathon prize, depending on venue).

Demo Plan — Live Run-of-Show

15.1 Suggested 3–4 minute flow

- **0:00–0:30** — Store dashboard: a dairy batch nearing expiry at its current static price.
- **0:30–1:00** — Trigger a simulated weather-drop / low-foot-traffic scenario and show the ML engine recompute the markdown live on screen — this is the single most important beat: it visually proves "dynamic," not "static."
- **1:00–1:45** — Fire the alert on a real phone in front of the judges: SMS/WhatsApp lands, reply "YES" or tap the claim link.
- **1:45–2:30** — Claim page opens: reservation countdown timer, Stripe/Razorpay test checkout, pickup-vs-delivery toggle, QR code generated.
- **2:30–3:00** — Second bulk batch: show the live descending-price auction ticking down over WebSocket — the visual "wow" moment.
- **3:00–3:30** — Cut to the NGO standing-order queue showing a shelter receiving the same batch type, free/subsidized, in parallel.
- **3:30–4:00** — Close on the Eco-Impact dashboard and a generated shareable impact receipt.

15.2 Risk mitigation

- Keep a pre-recorded backup video/GIF of the full flow in case venue Wi-Fi or Twilio delivery lags live.
- Use a seeded, reproducible demo dataset rather than relying on whatever real data happens to exist at demo time.
- Rehearse the "what if payment fails halfway through" question — be ready to explain the reserve-then-pay TTL and what happens on timeout (Chapter 8, Phase 3/5).

CHAPTER 16

Companion Deliverable — RULE.md (Agent Working Rules)

Per the four working-rule requirements — phase-scoped agent behavior, one branch per phase with frequent commits, explicit permission gates before pushing or starting new phases, and continuous documentation of what's built/in-progress/left plus technical decisions — a standalone **RULE.md** file has been generated alongside this PDF. It is meant to sit at the repository root so any developer or coding agent picks it up automatically before writing the first line of code.

File delivered separately: See *RULE.md*, provided alongside this document, for the full rule set, commit conventions, and the required *STATUS.md* / *DECISIONS.md* documentation structure.

End of document — ResQ-Chain Hackathon Playbook.